

Problem A – Special Identity Check mod 2

In the Binary Kingdom, all arithmetic is done modulo 2. You are given two square matrices A and B of size $n \times n$. Compute the matrix product:

$$C = A \times B \pmod{2},$$

where each element is computed as:

$$C_{i,j} = \left(\sum_{k=1}^n A_{i,k} \cdot B_{k,j} \right) \pmod{2}, \quad 1 \leq i, j \leq n.$$

After computing C , check if it is a **special identity matrix modulo 2**, defined by:

$$C_{i,j} = \begin{cases} 1 & \text{if } i = j \text{ or } i + j = n + 1, \\ 0 & \text{otherwise.} \end{cases}$$

Print YES if C satisfies this condition, otherwise print NO.

Input

- The first line contains an integer n ($1 \leq n \leq 100$).
- The next n lines each contain n integers describing matrix A .
- The following n lines each contain n integers describing matrix B .

Output

- Print YES if C is the special identity modulo 2, otherwise print NO.

Implementation Requirements

- Implement an iterative function to multiply A and B , applying modulo 2 after each sum.
- Implement a recursive function to verify the special identity matrix:
 - 1s on both main and anti-diagonal.
 - 0s elsewhere.

Example 1

n	3
A	$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$
B	$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$
Output	YES
Explanation	Multiplying $A \times B$ modulo 2 gives a matrix with 1s on both main and anti-diagonal, 0s elsewhere. Answer: YES.

Example 2

n	3
A	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
B	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Output	NO
Explanation	Multiplying $A \times B$ modulo 2 gives the regular identity matrix. Anti-diagonal elements are not all 1, so it does not satisfy the special identity. Answer: NO.

Example 3

n	2
A	$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$
B	$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$
Output	YES
Explanation	$C = A \times B \mod 2 = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$ which has 1s on both main and anti-diagonal and 0 elsewhere. Answer: YES.

Problem B – Circle Survival

In the ancient Kingdom of Circlonia, the royal council has summoned n brave warriors to a grand ceremonial arena. The king has devised a legendary trial to test courage and strategy: the warriors must stand in a circle, numbered 1 through n , and a mystical horn will signal the start of the ritual.

Starting from warrior number 1, the horn echoes every k beats. At each k -th beat, the corresponding warrior is removed from the circle and must leave the arena. The process continues, circling around the arena repeatedly, until only a single warrior remains.

The surviving warrior will be celebrated as the hero of Circlonia and receive the king's eternal favor.

Your task is to determine the position of this last remaining warrior (the survivor). Positions in the circle are numbered starting from 1.

Input Format

- Each test case consists of a single line containing two space-separated integers n and k ($1 \leq n, k \leq 10^5$) — the number of warriors and the elimination step.

Output Format

- For each test case, print a single integer — the position of the last remaining warrior.

Implementation Requirements

- Implement the solution using recursion.
- Your solution should efficiently handle up to 10^5 warriors.

Sample Test Cases

Input	5 2
Output	3
Explanation	Warriors are eliminated in the order: 2, 4, 1, 5. The last remaining warrior is at position 3.

Input	7 3
Output	4
Explanation	Elimination order: 3, 6, 2, 7, 5, 1. The survivor is at position 4.

Input	10 1
Output	10
Explanation	Every first warrior is eliminated in order: 1, 2, ..., 9. The last remaining warrior is at position 10.

Problem C – The Arcane Tower of Elements

In the mystical Kingdom of Eldoria, there stands a legendary Arcane Tower with n floors. Each floor contains a magical elemental crystal that can be either active or dormant.

A wizard has been tasked with activating all crystals in the tower following a mystical ritual. The ritual works floor by floor, starting from the ground floor. On floor i , the number of activations performed depends on all previous activations and a secret magical formula:

$$C_i = \left(\sum_{j=1}^{i-1} C_j \cdot j^2 + \lfloor \sqrt{i} \rfloor \right) \bmod m$$

Here, C_i is the number of elemental activations on floor i , and m is the magical modulus of the tower.

The wizard wishes to calculate the total magical power accumulated after activating all floors. Your task is to help the wizard compute this total magical power.

Input Format

- Each test case consists of a single line containing two space-separated integers n and m ($1 \leq n \leq 10^5$, $1 \leq m \leq 10^9$) — the number of floors and the magical modulus.

Output Format

- For each test case, print a single integer — the total magical power after activating all floors, modulo m .

Implementation Requirements

- Implement the solution using recursion.
- Efficiently handle up to 10^5 floors.
- Carefully compute each C_i modulo m to avoid overflow.

Sample Test Cases

Input	3 100
Output	14
Explanation	Compute C_1, C_2, C_3 using the given formula: $C_1 = (\lfloor \sqrt{1} \rfloor) \bmod 100 = 1$ $C_2 = (C_1 \cdot 1^2 + \lfloor \sqrt{2} \rfloor) \bmod 100 = (1 + 1) \bmod 100 = 2$ $C_3 = (C_1 \cdot 1^2 + C_2 \cdot 2^2 + \lfloor \sqrt{3} \rfloor) \bmod 100 = (1 + 2 \cdot 4 + 1) \bmod 100 = 10$ The total magical power is the sum of all activations squared: $1^2 + 2^2 + 10^2 = 1 + 4 + 100 = 105 \bmod 100 = 5$.

Problem D – The Alchemist’s Equation

In the ancient city of Numeria, an alchemist is attempting to create a mystical potion. To achieve the perfect balance, he must determine the quantities of three rare ingredients, x , y , and z , that satisfy the following system of linear equations:

$$\begin{cases} a_1x + b_1y + c_1z = d_1 \\ a_2x + b_2y + c_2z = d_2 \\ a_3x + b_3y + c_3z = d_3 \end{cases}$$

Your task is to help the alchemist by computing the solution of this system and reporting which of the following occurs:

- **Unique solution:** Print the values of x , y , and z rounded to 3 decimal places.
- **No solution:** Print `No solution`.
- **Infinitely many solutions:** Print `Infinitely many solutions`.

Input Format

- The first line contains an integer T ($1 \leq T \leq 100$) — the number of test cases.
- Each test case consists of 3 lines. Each line contains 4 space-separated integers: a_i b_i c_i d_i — the coefficients of x , y , z and the constant term for each equation.

Output Format

- For each test case, print one line as follows:
 - Three floating-point numbers x , y , z (rounded to 3 decimal places) if there is a unique solution.
 - `No solution` if the system has no solution.
 - `Infinitely many solutions` if there are infinitely many solutions.

Implementation Requirements

- Implement a recursive function to solve the above problem

Sample Test Cases

Input	1 1 2 -1 8 -2 1 3 -2 3 -1 -2 1
Output	0.417 3.083 -1.467
Explanation	Solving the system yields the unique solution $x = 0.417$, $y = 3.083$, $z = -1.467$.

Input	1 1 2 3 4 2 4 6 9 3 6 9 12
Output	No solution
Explanation	The second equation contradicts the linear combination of the others, so no solution exists.

Input	1 1 -1 2 3 2 -2 4 6 3 -3 6 9
Output	Infinitely many solutions
Explanation	All three equations are multiples of the first, so infinitely many solutions exist.

Problem E – Magic Series Expansion

In the kingdom of Numeria, wizards use infinite series to approximate magical functions like $\sin x$, $\cos x$, and e^x . Your task is to implement the ****Taylor series expansions**** of these functions for a given input x and number of terms n .

Series Expansions

$$\sin x = \sum_{k=0}^{n-1} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

$$\cos x = \sum_{k=0}^{n-1} (-1)^k \frac{x^{2k}}{(2k)!}$$

$$e^x = \sum_{k=0}^{n-1} \frac{x^k}{k!}$$

Input Format

- Each test case consists of a single line with: **function_name x n** where **function_name** is one of **sin**, **cos**, **exp**, x is a floating-point number, and n ($1 \leq n \leq 20$) is the number of terms in the series.

Output Format

- Print the computed value of the series rounded to 3 decimal places.

Implementation Requirements

- Implement the series recursion.
- Use factorial computation carefully to avoid integer overflow.
- Compare your result with standard library functions to verify accuracy.

Sample Test Cases

Input	sin 1.570796 10
Output	1.000000
Explanation	Approximating $\sin(\pi/2) \approx 1$ using 10 terms of its Taylor series.

Input	cos 3.141593 10
Output	-1.000
Explanation	Approximating $\cos(\pi) \approx -1$ using 10 terms.

Input	exp 1 10
Output	2.718
Explanation	Approximating e^1 using 10 terms.